

MANUAL TÉCNICO

Sistema de Ventas e Inventario

(Punto de Venta e Inventario — Java / Swing)

Repositorio: `PedroDonosoEPN/ProyectoProgramacion2doBimestre`

Proyecto de Programación — 2do Bimestre

Integrantes: Pedro Donoso, Eidan Clavijo, Mateo Cacuango, Matías Enríquez, Samuel Fuentes, Jhonatan Chango.

Fecha: julio 2026

Tabla de contenido

Tabla de contenido	2
1. Introducción.....	3
1.1 Objetivo del documento	3
1.2 Alcance.....	3
2. Tecnologías y herramientas utilizadas	3
3. Arquitectura del sistema	4
3.1 Paquete SistemaVentas (lógica de negocio)	4
3.2 Paquete GUI (interfaz de usuario).....	5
4. Estructura de carpetas del repositorio.....	6
5. Diagrama de clases del modelo de datos	6
6. Persistencia de datos	7
6.1 data/datos_inventario.txt	7
6.2 data/registro_ventas.txt	7
7. Flujo de funcionamiento	7
7.1 Inicio de la aplicación	7
7.2 Proceso de venta.....	7
7.3 Proceso de gestión de inventario	8
8. Control de acceso	8
9. Requisitos e instrucciones de instalación / ejecución	8
9.1 Requisitos previos	8
9.2 Clonar el repositorio	8
9.3 Ejecución desde Visual Studio Code.....	8
9.4 Ejecución por línea de comandos.....	8
10. Control de versiones (GitHub).....	9
11. Limitaciones conocidas y posibles mejoras	9
12. Conclusiones	9

1. Introducción

Este manual técnico documenta el diseño, la arquitectura y la implementación del Sistema de Ventas e Inventario, una aplicación de escritorio desarrollada en Java con interfaz gráfica Swing. El propósito de este documento es servir de referencia para el equipo de desarrollo y para cualquier persona que necesite mantener, ampliar o dar soporte al sistema en el futuro.

El sistema permite registrar ventas mediante un carrito de compras, descontar automáticamente el stock del inventario, generar comprobantes de pago (facturas) y administrar el catálogo de productos (agregar, editar y eliminar), todo ello a través de una interfaz gráfica moderna protegida con una clave de administrador para el módulo de inventario.

1.1 Objetivo del documento

Describir de forma clara y estructurada la arquitectura del software, los componentes que lo conforman, el modelo de datos, el flujo de ejecución y los requisitos técnicos necesarios para instalar, ejecutar y mantener la aplicación.

1.2 Alcance

- Módulo de Ventas: registro de productos vendidos, cálculo de totales, procesamiento de pago y generación de factura.
- Módulo de Inventario: gestión CRUD (agregar, editar, eliminar y consultar) del catálogo de productos, protegido por contraseña.
- Persistencia de datos en archivos de texto plano (sin motor de base de datos).
- Interfaz gráfica de escritorio construida sobre Java Swing.

2. Tecnologías y herramientas utilizadas

Componente	Tecnología	Detalle
Lenguaje	Java (100%)	Todo el proyecto está escrito en Java
Interfaz gráfica	Java Swing (AWT/Swing)	JFrame, JPanel, JTable, JDialog, componentes propios
Entorno de desarrollo	Visual Studio Code	Extensión Java (Language Support for Java, Java Debugger)
Gestión de dependencias	Ninguna (proyecto sin Maven/Gradle)	Compilación directa con javac / VS Code Java Projects
Persistencia	Archivos de texto (.txt)	data/datos_inventario.txt y data/registro_ventas.txt
Control de versiones	Git y GitHub	Repositorio remoto colaborativo del equipo

Componente	Tecnología	Detalle
Diseño / modelado	draw.io (archivos .dio / .drawio)	Carpeta design/: diagramas de casos de uso y de clases

3. Arquitectura del sistema

El sistema sigue una separación por responsabilidades similar a un patrón Modelo-Vista, organizada en dos paquetes principales de código fuente dentro de src/: SistemaVentas (lógica de negocio / modelo de datos) y GUI (interfaz de usuario). La vista invoca directamente a las clases del modelo; el modelo, a su vez, se encarga de la persistencia en archivos de texto.

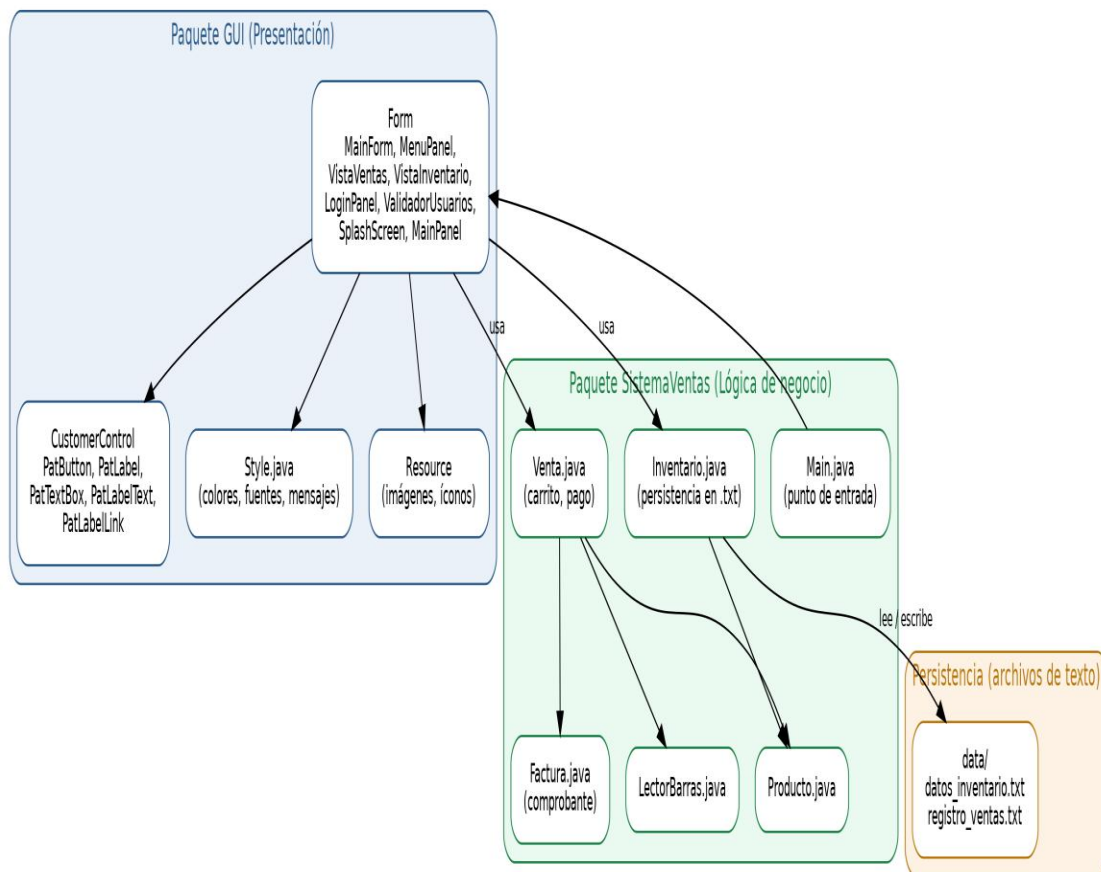


Figura 1. Arquitectura general por paquetes del sistema.

3.1 Paquete SistemaVentas (lógica de negocio)

Contiene las clases del dominio de la aplicación, independientes de la interfaz gráfica:

Clase	Responsabilidad
Main.java	Punto de entrada de la aplicación. Instancia MainForm dentro del hilo de eventos de Swing (EventQueue.invokeLater).
Producto.java	Representa un producto del inventario (nombre, código, cantidad, precio, descripción) y su serialización a texto (aTexto).
Inventario.java	Lee y escribe el catálogo de productos en data/datos_inventario.txt (listar, agregar, sobrescribir todo).
Venta.java	Administra el carrito de compras (clase interna ItemCarrito), calcula el total, procesa el pago y genera la factura.
Factura.java	Representa el comprobante de una venta: número, dinero recibido, vuelto y detalle; genera el texto del recibo.
LectorBarras.java	Simula la lectura de un código de barras a partir de la entrada de teclado (Scanner).

3.2 Paquete GUI (interfaz de usuario)

Contiene todo lo relacionado con la presentación, organizado en subpaquetes:

Subpaquete / Clase	Contenido
GUI.Form.MainForm	Ventana principal (JFrame). Fija tamaño (1100x700), ícono, y aloja el MenuPanel en el centro.
GUI.Form.MenuPanel	Menú principal con tarjetas interactivas: acceso a Módulo de Ventas y Control de Inventario (este último protegido con contraseña).
GUI.Form.VistaVentas	Panel del módulo de ventas: tabla del carrito, escaneo de producto, cálculo de total, cobro y generación de factura.
GUI.Form.VistaInventario	Panel del módulo de inventario: tabla de productos, diálogos para agregar/editar y eliminación de registros.
GUI.Form.LoginPanel / ValidadorUsuarios	Componentes de inicio de sesión y validación de clave de acceso (clave maestra para el módulo de inventario).
GUI.Form.SplashScreen / MainPanel	Pantalla de carga inicial y panel base reutilizable.
GUI.CustomerControl	Controles Swing personalizados y reutilizables: PatButton, PatLabel, PatTextBox, PatLabelText, PatLabelLink.
GUI.Style	Clase centralizada de estilos: colores, fuentes, bordes y utilidades de mensajes (showMsg, showMsgError, showConfirmYesNo).
GUI.Resource	Recursos gráficos: íconos e imágenes utilizadas en la interfaz (carrito, caja, logo, etc.).

4. Estructura de carpetas del repositorio

```
ProyectoProgramacion2doBimestre/
├── data/
│   ├── datos_inventario.txt      # Catálogo de productos (persistencia)
│   └── registro_ventas.txt       # Historial de ventas realizadas
├── design/
│   ├── 01UseClase.dio           # Diagrama de casos de uso
│   ├── DC.dio                  # Diagrama de clases
│   ├── DUC.dio                 # Diagrama de casos de uso (detalle)
│   └── UMLVentas.drawio        # Diagrama UML del módulo de ventas
├── src/
│   ├── GUI/
│   │   ├── CustomerControl/    # Controles Swing personalizados
│   │   ├── Form/              # Ventanas y paneles de la aplicación
│   │   ├── Resource/          # Imágenes e íconos
│   │   └── Style.java          # Estilos centralizados
│   └── SistemaVentas/
│       ├── Main.java           # Punto de entrada
│       ├── Producto.java
│       ├── Inventario.java
│       ├── Venta.java
│       ├── Factura.java
│       └── LectorBarras.java
├── .vscode/                    # Configuración del entorno VS Code
└── README.md
```

5. Diagrama de clases del modelo de datos

El siguiente diagrama resume los atributos y métodos principales de las clases del paquete SistemaVentas y su relación: Inventario gestiona objetos Producto; Venta agrupa varios ItemCarrito (cada uno referenciando un Producto) y, al procesar el pago, genera un objeto Factura.

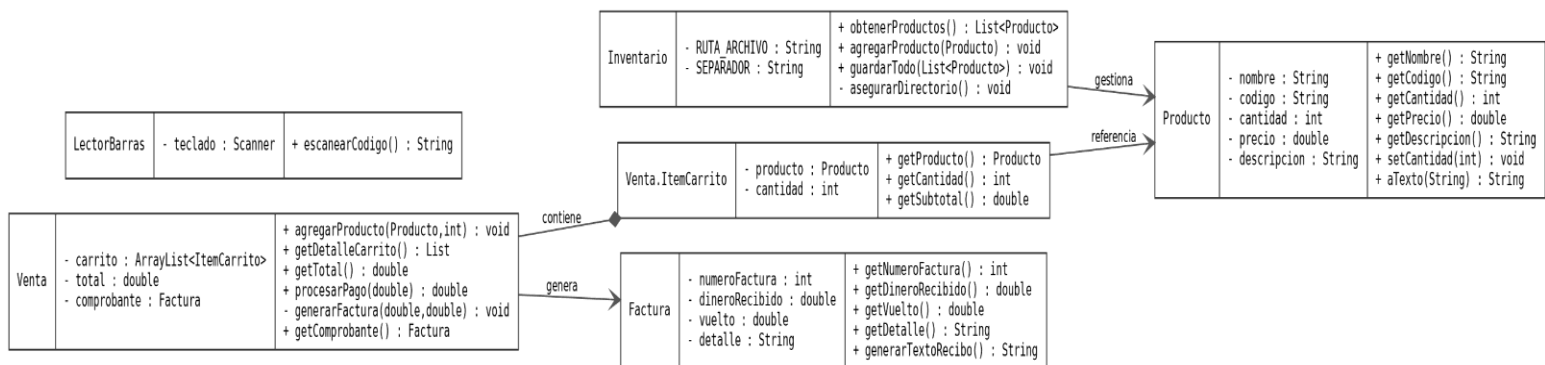


Figura 2. Diagrama de clases simplificado del modelo (paquete SistemaVentas).

Nota: en la carpeta design/ del repositorio se encuentran los diagramas UML originales elaborados por el equipo (casos de uso y clases) en formato draw.io, que pueden abrirse con la extensión Draw.io Integration en VS Code o en app.diagrams.net.

6. Persistencia de datos

El sistema no utiliza un motor de base de datos. En su lugar, la información se guarda en archivos de texto plano ubicados en la carpeta data/, con campos separados por comas (CSV simplificado).

6.1 data/datos_inventario.txt

Cada línea representa un producto, con el formato:

```
nombre,codigo,cantidad,precio,descripcion
```

Ejemplo real extraído del archivo del repositorio:

```
Paracetamol,MED-001,50,0.50,Tabletas 500mg  
Ibuprofeno,MED-002,30,2.50,Antiinflamatorio 400mg  
Amoxicilina,MED-003,20,3.50,Antibiotico 500mg
```

La clase Inventario.java es responsable de leer (obtenerProductos), agregar (agregarProducto, modo append) y reescribir completamente (guardarTodo) este archivo, creando automáticamente la carpeta data/ si no existe.

6.2 data/registro_ventas.txt

Cada línea representa una venta finalizada, con el formato:

```
fecha_hora_ISO;total;detalle_productos
```

Ejemplo real extraído del archivo del repositorio:

```
2026-07-18T17:47:41.339228100;0.50;Vendas x1  
2026-07-18T18:10:35.398569800;280.00;Arroz x5  
2026-07-18T18:14:21.328231;4.00;Ibuprofeno x1 | Alcohol x1 | Vendas x1
```

Este registro se genera desde el método registrarVentaArchivo() de la clase VistaVentas al completar una venta exitosamente.

7. Flujo de funcionamiento

7.1 Inicio de la aplicación

- Main.main() se ejecuta en el hilo de eventos de Swing y crea MainForm.
- MainForm centra la ventana (1100x700), fija el ícono y muestra MenuPanel.
- MenuPanel presenta dos tarjetas: “Módulo de Ventas” y “Control de Inventario”.

7.2 Proceso de venta

- El usuario entra al Módulo de Ventas desde el menú principal (acceso libre).
- VistaVentas permite escanear/ingresar el código del producto (escanearProducto).
- Cada producto agregado se añade al carrito (Venta.agregarProducto) y se recalcula el total.

- Al terminar la venta, se ingresa el efectivo recibido; `Venta.procesarPago` valida fondos y calcula el vuelto.
- Se descuenta el stock del inventario (`descontarStockInventario`) y se genera la Factura.
- La venta se registra en `data/registro_ventas.txt` (`registrarVentaArchivo`).

7.3 Proceso de gestión de inventario

- El usuario hace clic en “Control de Inventario” y se solicita una contraseña de administrador.
- Si la clave ingresada coincide con la definida en el sistema, se muestra `VistaInventario`.
- Desde ahí se pueden agregar, editar o eliminar productos; los cambios se guardan mediante `Inventario.guardarTodo` o `Inventario.agregarProducto`.

8. Control de acceso

El acceso al módulo de inventario está protegido mediante una contraseña de administrador solicitada en un cuadro de diálogo (`JPasswordField`) desde `MenuPanel.abrirModuloInventario()`. Existe además una clase auxiliar `ValidadorUsuarios` con el mismo propósito de validación.

Nota importante para el equipo: actualmente la contraseña está escrita directamente en el código fuente (hardcodeada) tanto en `MenuPanel` como en `ValidadorUsuarios`. Esto es aceptable para un proyecto académico, pero se recomienda documentarlo como una limitación conocida y, de ser posible, centralizar la validación en una sola clase para evitar duplicidad.

9. Requisitos e instrucciones de instalación / ejecución

9.1 Requisitos previos

- JDK (Java Development Kit) 8 o superior instalado y configurado en el PATH.
- Visual Studio Code con la extensión “Extension Pack for Java” (o cualquier IDE compatible con Java, como IntelliJ o Eclipse).
- Git para clonar el repositorio.

9.2 Clonar el repositorio

```
git clone https://github.com/PedroDonosoEPN/ProyectoProgramacion2doBimestre.git
```

9.3 Ejecución desde Visual Studio Code

- Abrir la carpeta del proyecto en VS Code.
- Abrir `src/SistemaVentas/Main.java`.
- Ejecutar con el botón “Run” que provee la extensión de Java, o presionar F5.

9.4 Ejecución por línea de comandos


```
# Compilar (desde la raíz del proyecto)
javac -d bin -sourcepath src src/SistemaVentas/Main.java

# Ejecutar
java -cp bin SistemaVentas.Main
```

Nota: al no usar Maven/Gradle, los recursos gráficos (carpeta src/GUI/Resource) deben copiarse también a la carpeta de salida (bin) para que los íconos se carguen correctamente; en VS Code esto ocurre automáticamente.

10. Control de versiones (GitHub)

El proyecto se aloja en el repositorio público de GitHub

PedroDonosoEPN/ProyectoProgramacion2doBimestre. Se recomienda que el equipo mantenga las siguientes prácticas:

- Realizar commits pequeños y descriptivos por funcionalidad.
- Usar ramas (branches) por integrante o por módulo (ventas, inventario, GUI) y fusionar mediante pull requests.
- Evitar subir archivos compilados (carpeta bin/) agregándolos al .gitignore.
- Actualizar el README.md del proyecto con la descripción real del sistema (actualmente contiene la plantilla por defecto de VS Code).

11. Limitaciones conocidas y posibles mejoras

Limitación actual	Mejora sugerida
Persistencia en archivos de texto plano	Migrar a una base de datos relacional (SQLite/MySQL) para mayor integridad y consultas
Contraseña de administrador hardcodeada	Externalizar en archivo de configuración cifrado o base de datos de usuarios
Sin manejo de concurrencia en archivos	Añadir bloqueo de archivo o migrar a base de datos si se usa en red
LoginPanel no integrado al flujo principal	Definir si se usará como pantalla de autenticación general o remover si es código muerto
Sin pruebas automatizadas (unit tests)	Incorporar JUnit para las clases del modelo (Producto, Inventario, Venta, Factura)

12. Conclusiones

El Sistema de Ventas e Inventario cumple con los requisitos funcionales básicos de un punto de venta: registro de ventas, control de stock y generación de comprobantes, todo mediante una interfaz gráfica clara y protegida por contraseña para las operaciones sensibles de inventario. Su arquitectura, aunque

simple, separa correctamente la lógica de negocio de la interfaz gráfica, lo que facilita su mantenimiento y una eventual migración a una base de datos real como mejora futura.